Abschlussprojekt SWP1 im SS 2025

Anleitung zur Dokumentation des Projektfortschritts mittels GitLab unter Berücksichtigung der besonderen Anforderung bei Nutzung von KI nach Meilenstein 1 (v1; Stand: 01.06.2025)

Nichtbeachtung der folgenden Hinweise kann zu einem Punktabzug bei der Bewertung des Abschlussprojekts bzw. im Plagiatsfall (insbesondere also bei der absichtlichen Verschleierung von KI-Verwendung) sogar zum Nichtbestehen der gesamten Veranstaltung führen.

Bitte beachten Sie, dass es jeder Gruppe freisteht, das Projekt ohne die Verwendung von KI umzusetzen. In diesem Fall wird die Bewertung ausschließlich nach klassischen Kriterien erfolgen (bspw. hinsichtlich der erwarteten Fähigkeiten im Bereich Programmieren an dem dieser Veranstaltung entsprechenden Punkt Ihres Studiums).

Selbstredend ist es, wie in den Präsenzveranstaltungen erläutert, dabei in jedem Fall (sprich: mit oder ohne KI) erforderlich und ein wichtiger Aspekt bei der Bewertung, dass Sie nicht nur die vorgegebenen, rudimentären Basisanforderungen des Abschlussprojekts (Erstellung der Ausgabe-CSV-Datei im geforderten Format und mit den geforderten Inhalten; Erstellung einer GUI zur Anzeige, Navigation und Filterung der Inhalte der betreffenden CSV-Datei; etc.) umsetzen, sondern **selbst kreativ zusätzliche Funktionen und Merkmale entwickeln und in das Projekt integrieren**.

Bei Gruppen, die KI verwenden, werden jedoch umso höhere Anforderungen an Originalität und Qualität dieser zusätzlichen Features gestellt, umso größer der Anteil der KI-Verwendung an Ihren Projektätigkeiten und -ergebnissen ist.

1. Einrichtung des KI-Log-Ordners

- Erstellen Sie im GitLab-Projektverzeichnis für jedes Gruppenmitglied einen separaten Ordner mit dem Namen: **`KI-Log-NUTZERNAME`**
  (Ersetzen Sie NUTZERNAME mit Ihrem HFT-Benutzernamen.)
- Dieser Ordner dient der strukturierten Ablage von KI-Prompt-Dokumentationen.

---

2. Erstellung der Dokumentationsdateien

- Für jeden Commit, der KI-generierten Code enthält, legen Sie eine separate Textdatei im entsprechenden Dokumentationsordner an.
  - **Format der Dateien:** Ausschließlich Textdateien (.txt), keine Word- bzw. PDF-Dokumente o. Ä.
  - **Nummerierung:** Jede KI-Dokumentationsdatei beginnt mit Ihren Initialen und wird dann fortlaufend nummeriert, z. B. `AX001.txt`, `AX002.txt` usw. bei der Gruppenteilnehmerin *Abigail Xylophon*.
- Dokumentieren Sie in der entsprechenden Datei jeweils die KI-Prompts, die Sie zur Generierung des Codes dieses Commits verwendet haben.

---

3. Promt-Dokumentation und Commit-Zuordnung bei Verwendung von KI

- **Maximale Anzahl an Prompts / maximale Gesamtgröße der Prompts pro Commit:** Jedem Commit dürfen maximal 10 Prompts bzw. maximal 8.000 „Prompt-Zeichen“ insgesamt (der **kleinere** der beiden Werte gilt jeweils) zugrunde liegen. (So soll besser nachvollziehbar bleiben, welche Teile des Codes durch welche Prompts neu erzeugt bzw. geändert wurden.)
- **Klarheit:** Die Prompts sind **klarer und nachvollziehbaren Form** aufzulisten (einschließlich verwendetem Modell, wie bspw. *Claude Sonnet 4*).
- **Commit-Zuordnung:**
    - **Die KI-Dokumentationsdatei mit Ihren jeweiligen Prompts muss natürlich selbst Teil des betreffenden Commits sein.** (sprich: Sie dürfen **nicht** separat erst die KI-erzeugten Codeänderungen committen und dann in einem weiteren Commit die zugehörige KI-Dokumentationsdatei einpflegen; dies würde die zuvor erwähnte Nachvollziehbarkeit der KI-erzeugten Codeänderungen deutlich erschweren)
    - Beschreiben Sie in der Commit-Message die entwickelten Funktionen/Merkmale und geben Sie zusätzlichen den Namen der zugehörigen KI-Dokumentationsdatei an.

---

4. Allgemeine (KI-unabhängige) Transparenz in der Teamarbeit

- **Nachvollziehbarkeit der Aufgabenverteilung:**
    - **Es muss eindeutig erkennbar sein, welches Teammitglied welche Teile des Projekts bearbeitet hat.** Dies gilt auch für die Verwendung von KI-Funktionen.
    - **Für jede Pair-Programming-Sessions** ist ein **separater Commit** durchzuführen. Die entsprechende **Commit-Message** muss dabei **klar beschreiben, wer an der betreffenden Pair-Programming-Session beteiligt war**. Dokumentieren Sie dabei auch möglichst weitgehend, wer in dieser Session welche Beiträge geliefert hat.

---

5. Beispielstruktur

```
Projektverzeichnis/
├── src/
│   ├── Main.java
│   ├── DataProcessor.java
│   └── GUIHandler.java
├── README.md
├── KI-Log-42xyab1bif/ (für die beispielhafte Nutzerin Abigail Xylophon mit HFT-Account 42xyab1bif)
│   ├── AX001.txt (im Commit mit Hash abc1234, siehe unten)
│   ├── AX002.txt (im Commit mit Hash def5678, siehe unten)
│   ├── AX003.txt (im Commit mit Hash ghi9012, siehe unten)
```

| └── AX004.txt (im Commit mit Hash jkl3456, siehe unten)

---

6. Vorgehen bei der Optimierung bestehenden Codes

- Wenn Sie bereits eigenen Code entwickelt haben (selbst wenn dieser noch nicht lauffähig ist):
  1. **Commiten Sie zuerst den bestehenden Code.**
     - **Commit-Message:** Beschreiben Sie den aktuellen Stand, z. B.

       "Erster Commit: Grundstruktur der GUI implementiert."

  2. **WENN Sie die KI zur Optimierung des Codes nutzen:**
     - Dokumentieren Sie die Prompts, die Sie für die Optimierung verwendet haben, in einer neuen Textdatei im KI-Log-Ordner.
  3. **Commiten Sie die optimierte Version des Codes zusammen mit der zugehörigen Textdatei.**
     - **Commit-Message:** Verweisen Sie auf die zugehörige Log-Datei, z. B.

       "Optimierung der Datenverarbeitung. [KI-Log: AX002.txt]"

---

7. Kreative Erweiterung des Projekts

- Nutzen Sie Ihre *Menschliche Intelligenz* und (sofern Sie sich dafür entscheiden sollten) auch die Möglichkeiten der *Künstlichen Intelligenz*, um neue Features und Funktionen zu entwickeln, insbesondere hinsichtlich der Benutzeroberfläche (GUI) und der automatisierten Erstellung optimierter Prüfungspläne.
  - **Beispiele für GUI-Erweiterungen:**
    - **Buttons:** Zusätzliche Schaltflächen für spezifische Aktionen.
    - **Filterfunktionen:** Mechanismen, um Inhalte basierend auf Benutzereingaben zu filtern.
    - **Sortieroptionen:** Sortieren von Daten in der GUI.
    - **Sprungoptionen:** Springen zum „Kollisionsblock“ von Veranstaltung A (sprich: wo diese in der Spalte Fach1 steht), falls in einem anderen Kollisionsblock (sprich: wo diese in Fach2 steht) auf diese geklickt wird.
    - **Statistiken:** Überlegen Sie, was für eine Person, die Ihr Tool zur Unterstützung der Prüfungsplanung einsetzt, nützlich sein könnte (bspw. zur Beurteilung der „Qualität“ des aktuellen Plans).
    - **Visuelle Anpassungen:** Layout, Hervorhebungen…
  - **Erstellung optimierter Prüfungspläne:**
    - Überlegen Sie sich, was einen „besseren“ Prüfungsplan ausmacht und was dazu benötigt wird.
    - Versehen Sie Ihr Programm mit einer Funktion zur Optimierung von Prüfungsplänen sowie mit damit verbundenen Features (bspw. dem Vergleich entsprechender Varianten).

- Hinweis: Die Optimierung von Plänen ist eine nicht-triviale Funktion und könnte für Sie ohne Verwendung von KI am aktuellen Punkt des Studiums „zu schwer“ sein. (wird bei Verzicht auf KI entsprechend auch nicht bei der Bewertung des Projekts erwartet)

Beispiele für Commits

1. Commit-Hash: abc1234

- **Commit-Message:**
  "Erstellung der Main.java-Klasse mit grundlegender Projektstruktur und Einstiegspunkt. [KI-Log: AX001.txt]"
- **Features:**
  - Eine main()-Methode als Einstiegspunkt für das Programm.
  - Initialisierung grundlegender Klassen.
- **Zugehörige KI-Log-Datei:** AX001.txt (ist Teil des Commits!)

---

2. Commit-Hash: def5678

- **Commit-Message:**
  "Implementierung der DataProcessor-Klasse mit grundlegenden Datenverarbeitungsfunktionen. [KI-Log: AX003.txt]"
- **Features:**
  - Funktion zur Berechnung der Summe einer Liste von Zahlen.
  - Funktion zur Entfernung von Duplikaten aus einer Liste.
  - JavaDoc-Kommentare zur Dokumentation der Funktionen.
- **Zugehörige KI-Log-Datei:** AX002.txt (ist Teil des Commits!)

---

3. Commit-Hash: ghi9012

- **Commit-Message:**
  "Entwicklung der GUIHandler-Klasse mit grundlegender Benutzeroberfläche. [KI-Log: AX003.txt]"
- **Features:**
  - Erstellung eines Fensters mit Java Swing.
  - Hinzufügen eines Textfeldes für Benutzereingaben.
  - Schaltfläche für die Übertragung von Benutzereingaben an die Konsole.
- **Zugehörige KI-Log-Datei:** AX003.txt (ist Teil des Commits!)

---

4. Commit-Hash: jkl3456

- **Commit-Message:**
  "Erweiterung der GUIHandler-Klasse: Hinzufügen eines Buttons und einer Filterfunktion. [KI-Log: AX004.txt]"
- **Features:**
  - Zusätzlicher Button in der GUI, der eine Filterfunktion aktiviert.
  - Filterfunktion, die es ermöglicht, Einträge basierend auf einem Suchkriterium zu filtern (z. B. Zahlen > 10).
  - Anzeige der gefilterten Ergebnisse in der Benutzeroberfläche.
- **Zugehörige KI-Log-Datei:** AX004.txt (ist Teil des Commits!)